

// LOCAL INFERENCE · DEVELOPER NOTES

Ollama

A lightweight runtime for downloading, serving, and running open-weight large language models entirely on your own machine — no cloud, no API key, full control.

CLI + APIINTERFACESGGUFMODEL FORMATCPU / GPUEXECUTIONREST :11434LOCAL SERVER

01 Topic Overview



Ollama Core

A CLI + background server that manages local LLMs like a package manager manages libraries.



Modelfile

A Dockerfile-style spec for building custom models — prompts, params, and base weights.



Quantization

Compresses model weights (e.g. Q4_K_M) so large models run on consumer hardware.



llama.cpp Engine

The C/C++ inference backend under the hood — fast, portable, GPU-accelerated.



REST API

Every model is instantly callable over HTTP at localhost:11434.



Model Library

A registry of pre-quantized open models — Llama, Mistral, Gemma, Qwen, and more.

02 Major Concepts

01What Is Ollama

CORE

DEFINITION

Ollama is an open-source runtime that packages an LLM's weights, configuration, and tokenizer into a single distributable unit, then serves it locally through a CLI and a REST API — abstracting away CUDA drivers, tokenizer setup, and prompt templating.

KEY CONCEPTS

- Runs as a persistent background daemon (ollama serve)
- Pulls models from its own model registry, similar to Docker Hub
- Handles GPU offloading automatically when hardware is available
- Keeps models resident in memory between requests for speed

02The Modelfile

CONFIG

DEFINITION

A plain-text build spec — modeled after a Dockerfile — describing which base model to use, the system prompt, sampling parameters, and a chat template. It's how you version and share a customized model configuration.

WHY IT MATTERS

It turns "prompt engineering + settings" into a reproducible artifact you can build once with ollama create and reuse anywhere, without re-typing system prompts each session.

03Quantization & GGUF

PERFORMANCE

DEFINITION

Quantization reduces the numerical precision of a model's weights (e.g. 16-bit → 4-bit) to shrink file size and memory footprint. Ollama distributes models in the GGUF format, the binary container used by the llama.cpp engine.

KEY CONCEPTS

- Common levels: Q4_K_M, Q5_K_M, Q8_0 — trade accuracy for size
- Lower bit-depth = smaller RAM/VRAM need, slightly lower quality
- A 7B model at Q4 fits in roughly 4–5 GB of memory

 ANALOGY

Think of Ollama like **Docker for language models**. A "model" is like an image — you pull it, it's stored in layers, and you run it locally. The Modelfile is Ollama's version of a Dockerfile, and the local registry works just like Docker Hub — except each layer is a chunk of neural network weights instead of a filesystem diff.

04

 The Inference Engine (llama.cpp)

ENGINE

DEFINITION

Under the hood, Ollama wraps llama.cpp — a dependency-free C/C++ inference engine that executes GGUF models directly on CPU or GPU. It handles tokenization, KV-cache management, batched decoding, and hardware-specific kernels (Metal, CUDA, ROCm, AVX2).

WHY IT MATTERS

Because the engine is written close to the metal with no Python overhead, Ollama can serve models with low latency on ordinary laptops — something a full PyTorch stack struggles to do efficiently.

05

 REST API & Client Libraries

INTEGRATION

DEFINITION

The moment ollama serve is running, every pulled model is reachable over HTTP at http://localhost:11434 — with endpoints for chat, generate, embeddings, and model management.

EXAMPLE

Any HTTP client — curl, Python's requests, or the official ollama Python/JS packages — can call it, making it a drop-in local replacement for a cloud model API during development.

06

 Context Window & Memory

RESOURCES

DEFINITION

The context window is the maximum number of tokens (prompt + history + response) a model can attend to at once. Ollama exposes this via the num_ctx parameter, and it directly drives how much RAM/VRAM the KV-cache consumes — a larger context window means more memory reserved per loaded model.

⚡ KEY POINT

Model size (parameters) and quantization level are the two biggest levers over both **quality** and **hardware requirements**. Bigger + higher precision = better answers but heavier memory demand; smaller + lower bit-depth = faster and lighter, with some quality loss.

⚠ COMMON MISTAKE

Pulling a model that's larger than available RAM/VRAM causes Ollama to spill to disk swap or fail to load. Always check a model's minimum memory requirement against your machine before pulling — a 70B model typically needs 40GB+ even at Q4 quantization.

BASH · CLI BASICS

```
# download and run a model interactively
ollama run llama3.1

# pull a model without starting a chat session
ollama pull mistral:7b

# list all locally installed models
ollama list

# remove a model to free disk space
ollama rm mistral:7b

# show running model processes + memory usage
ollama ps
```

DOCKERFILE · CUSTOM MODELFILE

```
FROM llama3.1:8b

PARAMETER temperature 0.7
PARAMETER num_ctx 8192

SYSTEM ""You are a concise senior backend engineer.
Answer in short, direct technical language.""
```

BASH · BUILD & SERVE

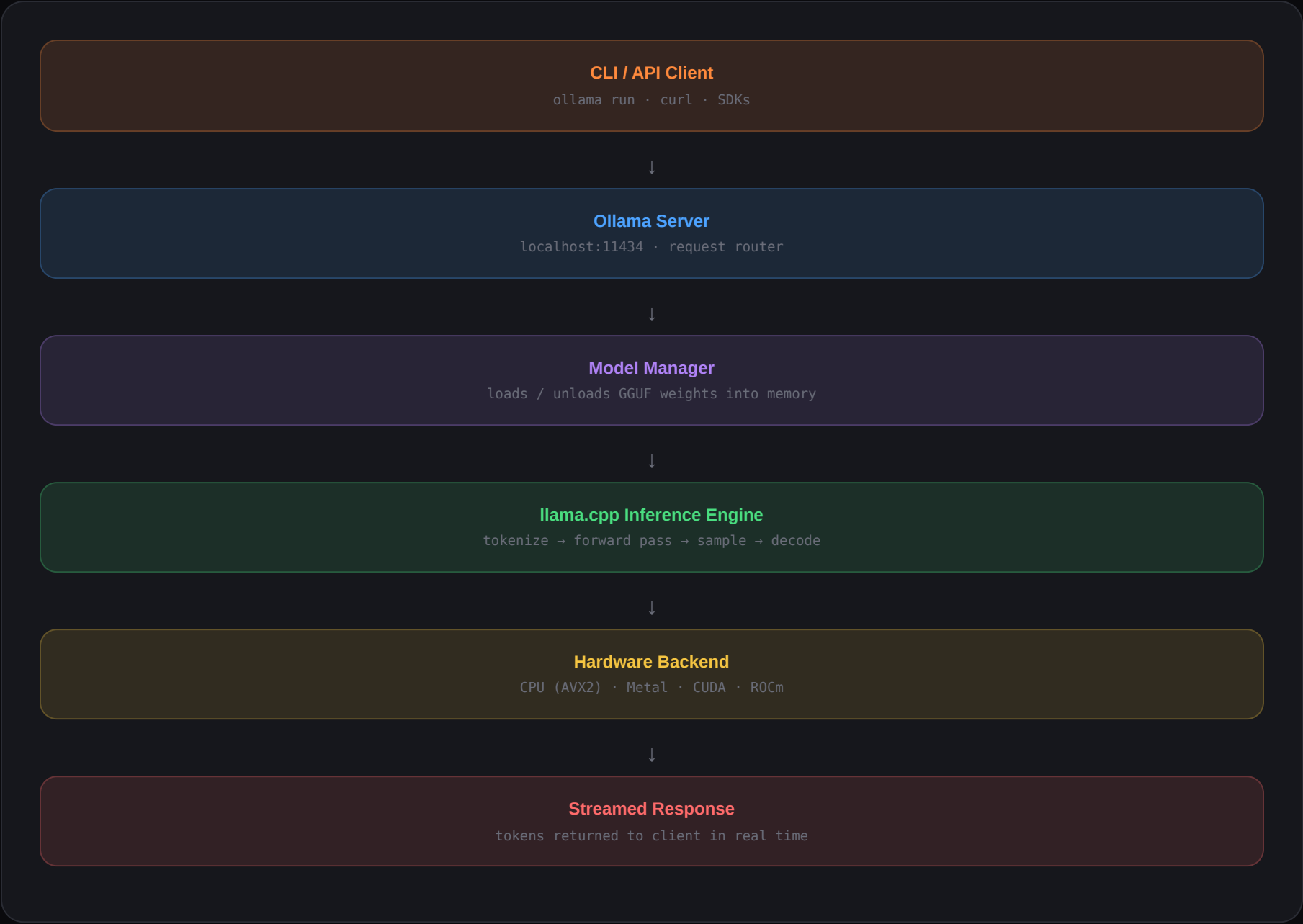
```
# build a custom model from a Modelfile
ollama create backend-bot -f ./Modelfile

# start the local API server
ollama serve
```

JSON · REST API CALL

```
curl http://localhost:11434/api/chat -d '{
  "model": "llama3.1",
  "messages": [{ "role": "user", "content": "Explain quantization" }],
  "stream": false
}'
```

05 Workflow / Architecture



06 Ollama vs. Alternatives

APPROACH	SETUP EFFORT	LOCAL CONTROL	BEST FOR
Ollama	Low	Full	Quick local dev, prototyping, offline use
Raw llama.cpp	High	Full	Fine-grained performance tuning
LM Studio	Medium	Full	GUI-first, non-CLI users
Cloud API (OpenAI etc.)	Low	None	Largest frontier models, no local hardware

07 Step-by-Step Implementation

- 1 Install Ollama

Download the installer for macOS/Windows or run `curl -fsSL https://ollama.com/install.sh | sh` on Linux. This installs the CLI and registers the background service.
- 2 Pull a base model

Run `ollama pull llama3.1` to download quantized weights from the model registry to `~/.ollama/models`.
- 3 Chat interactively

Run `ollama run llama3.1` to drop into a REPL-style chat session directly in the terminal.
- 4 Customize with a Modelfile

Write a Modelfile with a system prompt and parameters, then run `ollama create mybot -f Modelfile` to save it as a new named model.
- 5 Call it from your app

Send requests to `POST /api/chat` on `localhost:11434`, or use the official Python/JS client libraries to integrate it into a real application.

08 Final Summary

// WRAP - UP

Ollama turns running an LLM into a one-line command by bundling a quantized GGUF model, a llama.cpp inference engine, and a local REST API into a single tool — giving developers cloud-API convenience with fully offline, private execution.



Package-manager UX

Pull, run, and remove models like npm or apt packages.



Modelfile customization

Reproducible, versioned model configs like a Dockerfile.



Fully local & private

No data leaves the machine — no API keys, no network calls.



Instant REST API

Every model is callable over HTTP the moment it's pulled.

09 Quick Revision Cards

WHAT PORT DOES OLLAMA SERVE ON?

11434 — the default local REST API port.

WHAT FORMAT ARE OLLAMA MODELS STORED IN?

GGUF, a compact binary format used by the llama.cpp engine.

WHAT IS A MODELFILE?

A Dockerfile-style spec defining base model, params, and system prompt.

WHAT DOES QUANTIZATION TRADE OFF?

Smaller memory footprint and faster inference vs. slightly lower output quality.

WHICH ENGINE POWERS INFERENCE?

llama.cpp — a dependency-free C/C++ backend with GPU acceleration.

COMMAND TO LIST LOCAL MODELS?

`ollama list`

COMMAND TO SEE RUNNING/LOADED MODELS?

`ollama ps`

WHAT CONTROLS CONTEXT LENGTH?

The `num_ctx` parameter — bigger values need more memory.

#local-llm

#ollama

#llama.cpp

#gguf

#on-device-ai

#developer-notes